EECE 2323 Digital Logic Design Lab

Name: Arefin Mahmood **Lab Report 3: Adding Register File to ALU**
Date: 06/01/2026

Background & Purpose

In this lab, I added a register file to the ALU datapath that I built in Lab 2. In the last lab, the ALU could already do operations like ADD, AND, OR, XOR, and branch checks. The main difference in this lab is that the ALU inputs now come from stored register values instead of only direct inputs. I had to write values into the register file, read them back out, send them through muxes, and then use them as the ALU inputs.

This connects more closely to how a processor datapath works. The ALU does the actual operation, but the register file holds the values that the ALU works with. This lab also introduced sequential logic because the register file uses a clock. The ALU output changes based on its current inputs, but the register file stores values and only updates when writing is enabled on a clock edge.

Design Analysis

The main change from Lab 2 was adding the register file before the ALU. Instead of only using direct VIO inputs, this design lets me store values first and then read them back into the ALU. The register file has 8 registers, and each register is 16 bits wide. It uses RegWrite, regfile_WriteAddress, and regfile_WriteData to save values. The two read addresses, regfile_ReadAddress1 and regfile_ReadAddress2, choose which two register values are sent out.

I also used two 16-bit 2-to-1 muxes before the ALU inputs. Mux1 controls alu_input1. When ALUSrc1 = 0, alu_input1 comes from regfile_ReadData1. When ALUSrc1 = 1, it comes from the zero register. Mux2 controls alu_input2. When ALUSrc2 = 0, alu_input2 comes from regfile_ReadData2. When ALUSrc2 = 1, it comes from alu_input2_instr_src.

The register file is the sequential part of the circuit, so it needs the clock. Writing only happens on the clock edge when RegWrite is 1. The read outputs are combinational, so changing the read address updates the read data without waiting for another write. This setup let me write 000C into register 2, write 0011 into register 3, and then read both values into the ALU for testing.

Results and Analysis

The simulation worked as expected. I reset the register file, wrote 000C into register 2, and wrote 0011 into register 3. Then I read both registers and sent them into the ALU. With ALUOp = 0, the ALU output was 001D. This is correct because 000C is 12, 0011 is 17, and 12 + 17 = 29.

I also tested a few other ALU operations with the same inputs. For AND, ALUOp = 2 and the output was 0000. For OR, ALUOp = 3 and the output was 001D. For BNEZ, ALUOp = 7 and take_branch became 1 because alu_input1 was 000C, which is not zero.

In hardware, I used VIO to write and read the register file. The final test showed alu_input1 = 000C, alu_input2 = 0011, and alu_output = 001D. The VIO values did not always update right away, so I had to press Enter after changing values and check the signals carefully. Once the values updated, the hardware result matched the simulation.

Conclusion & Recommendations

In this lab, I learned how to connect a register file to the ALU and make the datapath feel more like a small processor. The register file stored the values, the muxes chose what went into the ALU, and

ALUOp selected the operation. The main test worked the way I expected: I stored 000C in register 2, stored 0011 in register 3, read both values back, and added them to get 001D.

This lab also made the difference between combinational and sequential logic clearer. The ALU and muxes update based on the current inputs, but the register file has to wait for the clock to write data. That is why RegWrite and the clock edge matter when saving values into registers.

For future labs, I would use a simple test table before going into VIO. There were a lot of signals to keep track of, so it was easier when I tested one part at a time: writing to the register file, reading the values back, checking the mux inputs, and then changing ALUOp for the ALU tests.

Appendix A: Prelab and Test Data/Simulation

Table A1: Prelab Table for Register File and ALU Operations

Description	ALUSrc1	ALUSrc2	ALUOp	regfile_ReadAddress1	regfile_ReadAddress2	regfile_WriteAddress	regfile_WriteData	RegWrite	regfile_ReadData1	regfile_ReadData2	alu_input1	alu_input2	alu_output
Add zero register and immediate value of 12	1	1	0	0	X	X	12	0	0	X	0	12	12
Write the value from the previous step to address 2	X	X	X	X	X	2	12	1	X	X	X	X	X
Add zero register and immediate value of 17	1	1	0	0	X	X	17	0	0	X	0	17	17
Write the value from the previous step to address 3	X	X	X	X	X	3	17	1	X	X	X	X	X
Add values in address 2 and address 3	0	0	0	2	3	X	X	0	12	17	12	17	29

Appendix B: Simulation Evidence

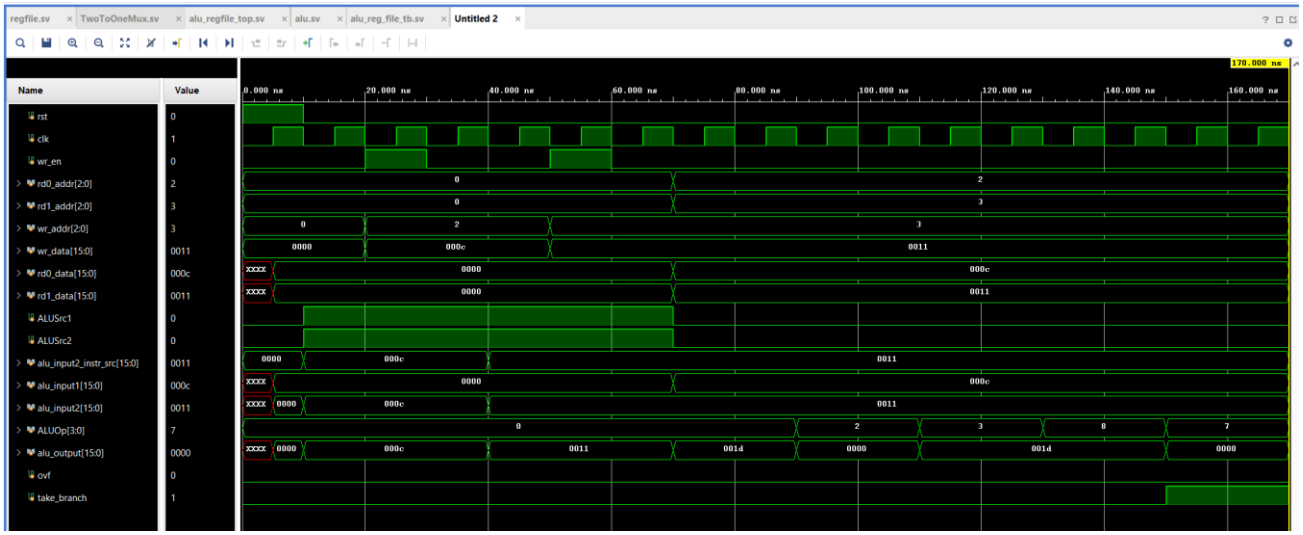
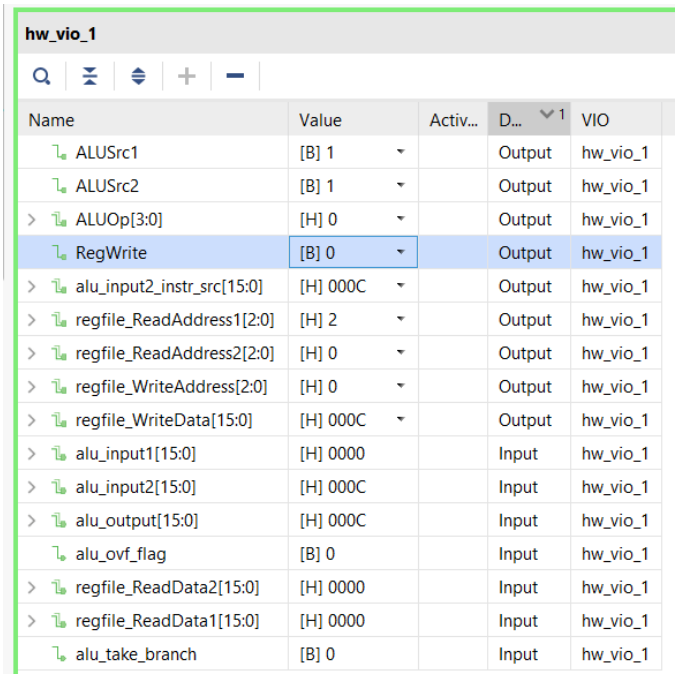


Figure B1: Full Behavioral Simulation Waveform

Appendix C: Hardware Evidence



hw_vio_1

Name	Value	Activ...	D...	VIO
ALUSrc1	[B] 1		Output	hw_vio_1
ALUSrc2	[B] 1		Output	hw_vio_1
ALUOp[3:0]	[H] 0		Output	hw_vio_1
RegWrite	[B] 0		Output	hw_vio_1
alu_input2_instr_src[15:0]	[H] 000C		Output	hw_vio_1
regfile_ReadAddress1[2:0]	[H] 2		Output	hw_vio_1
regfile_ReadAddress2[2:0]	[H] 0		Output	hw_vio_1
regfile_WriteAddress[2:0]	[H] 0		Output	hw_vio_1
regfile_WriteData[15:0]	[H] 000C		Output	hw_vio_1
alu_input1[15:0]	[H] 0000		Input	hw_vio_1
alu_input2[15:0]	[H] 000C		Input	hw_vio_1
alu_output[15:0]	[H] 000C		Input	hw_vio_1
alu_ovf_flag	[B] 0		Input	hw_vio_1
regfile_ReadData2[15:0]	[H] 0000		Input	hw_vio_1
regfile_ReadData1[15:0]	[H] 0000		Input	hw_vio_1
alu_take_branch	[B] 0		Input	hw_vio_1

Figure C1: VIO Register Write and Read Test Showing Register 2 = 000C

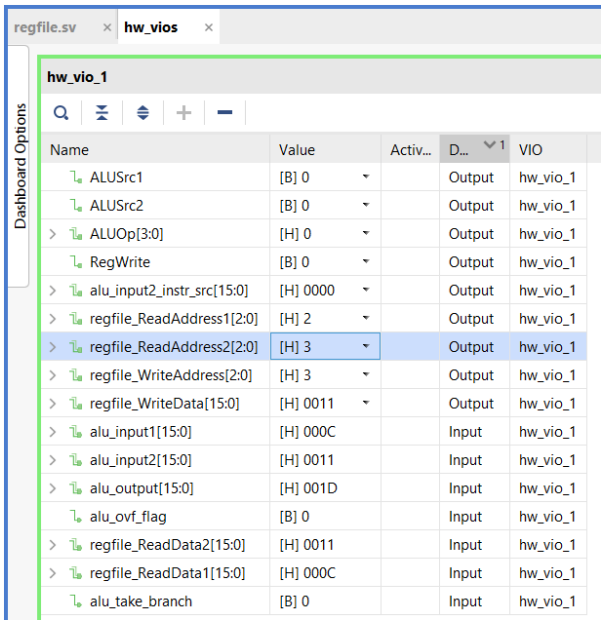


Figure C2: VIO Register Write and Read Test Showing Register 3 = 0011

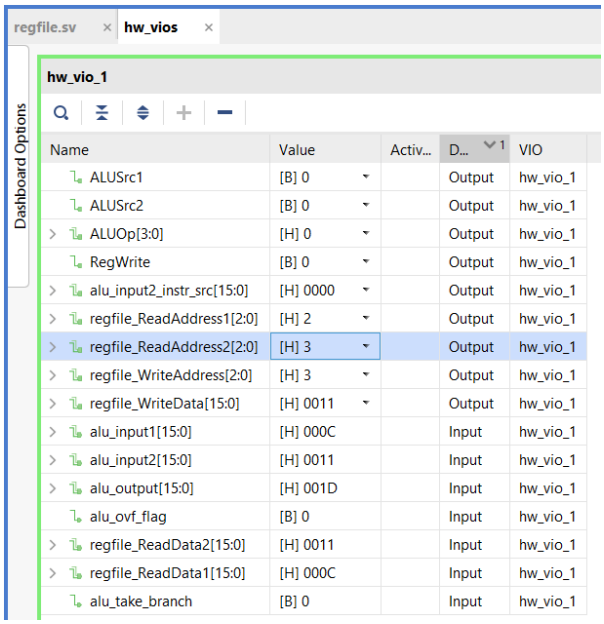


Figure C3: ADD Test Showing alu_input1 = 000C, alu_input2 = 0011, and alu_output = 001D

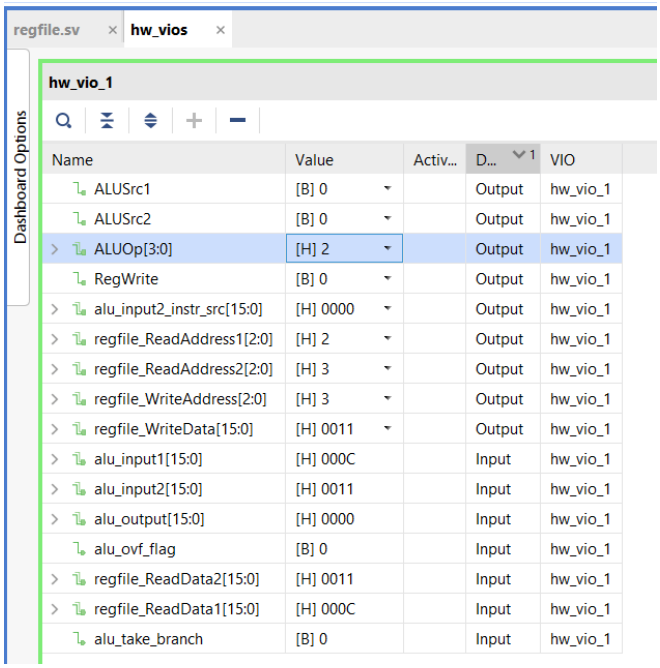


Figure C4: VIO AND Test Showing ALUOp = 2 and alu_output = 0000

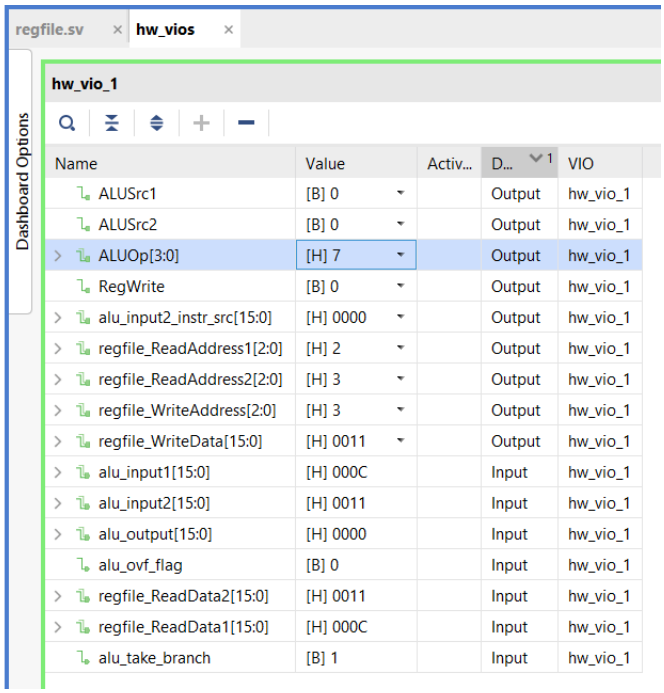


Figure C5: VIO BNEZ Test Showing take_branch = 1

Appendix D: Register File Code

regfile.sv

```
module regfile(
    input logic rst,                // reset signal all registers to 0
```

```
    input logic clk,                // clock signal
    input logic wr_en,              // rite enable
    input logic [2:0] rd0_addr,     // address for first read port
    input logic [2:0] rd1_addr,     // address for second read port
    input logic [2:0] wr_addr,      // address for write port
    input logic [15:0] wr_data,     // data that will be written into the register file
    output logic [15:0] rd0_data,   // read from rd0_addr
    output logic [15:0] rd1_data    //read from rd1_addr
);

    //creates 8 registers.each register stores 16 bits.
    logic [15:0] registers [0:7];
    integer i;

    // block handles reset and writing
    // It only changes values on the rising edge of the clock
    always_ff @(posedge clk) begin

        // if reset is on, clear all registers to 0.
        if (rst) begin
            for (i = 0; i < 8; i = i + 1) begin
                registers[i] <= 16'd0;
            end
        end

        // if reset is off and write enable is on, write data.
        else if (wr_en) begin
            registers[wr_addr] <= wr_data;
        end
    end

    // these are the two read outputs. they show the values stored at the selected read addresses.
    assign rd0_data = registers[rd0_addr];
```

```
    assign rd1_data = registers[rd1_addr];
endmodule
```

Appendix E: Two-to-One Mux Code

```
module TwoToOneMux(
    input logic [15:0] a,        // First input. Chosen when sel_mux = 0.
    input logic [15:0] b,        // Second input. Chosen when sel_mux = 1.
    input logic sel_mux,         // Select signal for the mux.
    output logic [15:0] out      // Output of the mux.
);

    // If sel_mux is 0, out = a.
    // If sel_mux is 1, out = b.
    assign out = sel_mux ? b : a;
endmodule
```

Appendix F: ALU Code

```
`timescale 1ns / 1ps

module alu(
    input  logic signed [15:0] a,
    input  logic signed [15:0] b,
    input  logic [3:0] s,
    output logic signed [15:0] f,
    output logic ovf,
    output logic take_branch
);
    always_comb begin
        // Default values
        f = 16'h0000;
        ovf = 1'b0;
        take_branch = 1'b0;
```

```
case (s)
    // ADD: f = a + b
    4'b0000: begin
        f = a + b;

        // Overflow happens when a and b have the same sign,
        // but the result has a different sign.
        if ((a[15] == b[15]) && (f[15] != a[15]))
            ovf = 1'b1;
        else
            ovf = 1'b0;

        take_branch = 1'b0;
    end

    // INV: f = bitwise inversion of b
    4'b0001: begin
        f = ~b;
        ovf = 1'b0;
        take_branch = 1'b0;
    end

    // AND: f = a AND b
    4'b0010: begin
        f = a & b;
        ovf = 1'b0;
        take_branch = 1'b0;
    end

    // OR: f = a OR b
    4'b0011: begin
        f = a | b;
        ovf = 1'b0;
```

```
    take_branch = 1'b0;
end

// SRA: arithmetic right shift
4'b0100: begin
    f = a >>> b;
    ovf = 1'b0;
    take_branch = 1'b0;
end

// SLL: logical left shift
4'b0101: begin
    f = a << b;
    ovf = 1'b0;
    take_branch = 1'b0;
end

// BEQZ: branch if a equals zero
4'b0110: begin
    f = 16'h0000;
    ovf = 1'b0;

    if (a == 16'h0000)
        take_branch = 1'b1;
    else
        take_branch = 1'b0;
end

// BNEZ: branch if a does not equal zero
4'b0111: begin
    f = 16'h0000;
    ovf = 1'b0;
```

```
            if (a != 16'h0000)
                take_branch = 1'b1;
            else
                take_branch = 1'b0;
        end

        // XOR: f = a XOR b
        4'b1000: begin
            f = a ^ b;
            ovf = 1'b0;
            take_branch = 1'b0;
        end

        // Any unused selector value
        default: begin
            f = 16'h0000;
            ovf = 1'b0;
            take_branch = 1'b0;
        end
    endcase
  end
endmodule
```

Appendix G: Top Module Code

```
`timescale 1ns / 1ps

module alu_regfile_top(
    input logic clk,    // clock for vio and regfile
    input logic reset,
    output logic ovf_ctrl,
    output logic take_branch,
    output logic [3:0] disp_en,      // 7-segment display enable
```

```
    output logic [6:0] seg7_output   // 7-segment output
);

logic [15:0] alu_input1, alu_input2, alu_input2_instr_src;
logic [15:0] alu_output;
logic [3:0] ALUOp;
logic RegWrite;
logic alu_ovf_flag, alu_take_branch;

logic [2:0] regfile_ReadAddress1;    // source register 1 address
logic [2:0] regfile_ReadAddress2;    // source register 2 address
logic [2:0] regfile_WriteAddress;    // destination register address
logic [15:0] regfile_WriteData;      // write data
logic [15:0] regfile_ReadData1;      // source register 1 data
logic [15:0] regfile_ReadData2;      // source register 2 data
logic ALUSrc1, ALUSrc2;
logic [15:0] zero_register = 0;

assign ovf_ctrl = alu_ovf_flag;
assign take_branch = alu_take_branch;

// register file
regfile reg1(
    .rst(reset),
    .clk(clk),
    .wr_en(RegWrite),
    .rd0_addr(regfile_ReadAddress1),
    .rd1_addr(regfile_ReadAddress2),
    .wr_addr(regfile_WriteAddress),
    .wr_data(regfile_WriteData),
    .rd0_data(regfile_ReadData1),
    .rd1_data(regfile_ReadData2)
);
```

```
// mux for alu input 1
TwoToOneMux mux1(
    .a(regfile_ReadData1),
    .b(zero_register),
    .sel_mux(ALUSrc1),
    .out(alu_input1)
);

// mux for alu input 2
TwoToOneMux mux2(
    .a(regfile_ReadData2),
    .b(alu_input2_instr_src),
    .sel_mux(ALUSrc2),
    .out(alu_input2)
);

// alu from lab 2
alu sixteenbit_alu(
    .s(ALUOp),
    .a(alu_input1),
    .b(alu_input2),
    .f(alu_output),
    .ovf(alu_ovf_flag),
    .take_branch(alu_take_branch)
);

// seven segment display
Adaptor_display display(
    .clk(clk),
    .input_value(alu_output),
    .disp_en(disp_en),
    .seg7_output(seg7_output)
```

```
    );

    // vio core
    vio_0 vio (
        .clk(clk),

        .probe_in0(alu_output),
        .probe_in1(alu_ovf_flag),
        .probe_in2(alu_take_branch),
        .probe_in3(regfile_ReadData1),
        .probe_in4(regfile_ReadData2),
        .probe_in5(alu_input1),
        .probe_in6(alu_input2),

        .probe_out0(alu_input2_instr_src),
        .probe_out1(ALUOp),
        .probe_out2(ALUSrc1),
        .probe_out3(ALUSrc2),
        .probe_out4(RegWrite),
        .probe_out5(regfile_ReadAddress1),
        .probe_out6(regfile_ReadAddress2),
        .probe_out7(regfile_WriteAddress),
        .probe_out8(regfile_WriteData)
    );

endmodule
```

Appendix H: Testbench Code

```
module alu_reg_file_tb;
    logic rst;
    logic clk;
    logic wr_en;
```

```
logic [2:0] rd0_addr;
logic [2:0] rd1_addr;
logic [2:0] wr_addr;
logic [15:0] wr_data;
logic [15:0] rd0_data;
logic [15:0] rd1_data;

logic ALUSrc1;
logic ALUSrc2;

logic [15:0] alu_input2_instr_src;

logic signed [15:0] alu_input1;
logic signed [15:0] alu_input2;
logic [3:0] ALUOp;
logic signed [15:0] alu_output;
logic ovf;
logic take_branch;

// clock for the register file
always begin
    #5 clk = ~clk;
end

// register file
regfile reg1 (
    .rst(rst),
    .clk(clk),
    .wr_en(wr_en),
    .rd0_addr(rd0_addr),
    .rd1_addr(rd1_addr),
    .wr_addr(wr_addr),
    .wr_data(wr_data),
```

```
    .rd0_data(rd0_data),
    .rd1_data(rd1_data)
);

// mux for alu input 1
TwoToOneMux mux1 (
    .a(rd0_data),
    .b(16'd0),
    .sel_mux(ALUSrc1),
    .out(alu_input1)
);

// mux for alu input 2
TwoToOneMux mux2 (
    .a(rd1_data),
    .b(alu_input2_instr_src),
    .sel_mux(ALUSrc2),
    .out(alu_input2)
);

// alu from lab 2
alu sixteenbit_alu (
    .a(alu_input1),
    .b(alu_input2),
    .s(ALUOp),
    .f(alu_output),
    .ovf(ovf),
    .take_branch(take_branch)
);

initial begin

    // starting values
```

```
clk = 0;
rst = 1;
wr_en = 0;
rd0_addr = 3'd0;
rd1_addr = 3'd0;
wr_addr = 3'd0;
wr_data = 16'd0;
ALUSrc1 = 0;
ALUSrc2 = 0;
alu_input2_instr_src = 16'd0;
ALUOp = 4'b0000;

#10;

rst = 0;

// make 12 using zero + immediate
rd0_addr = 3'd0;
ALUSrc1 = 1;
ALUSrc2 = 1;
alu_input2_instr_src = 16'd12;
ALUOp = 4'b0000;
#10;

// save 12 in register 2
wr_addr = 3'd2;
wr_data = 16'd12;
wr_en = 1;
#10;

wr_en = 0;
#10;
```

```
// make 17 using zero + immediate
rd0_addr = 3'd0;
ALUSrc1 = 1;
ALUSrc2 = 1;
alu_input2_instr_src = 16'd17;
ALUOp = 4'b0000;
#10;

// save 17 in register 3
wr_addr = 3'd3;
wr_data = 16'd17;
wr_en = 1;
#10;

wr_en = 0;
#10;

// read register 2 and 3, then add
rd0_addr = 3'd2;
rd1_addr = 3'd3;
ALUSrc1 = 0;
ALUSrc2 = 0;
ALUOp = 4'b0000;
#20;

// AND
ALUOp = 4'b0010;
#20;

// OR
ALUOp = 4'b0011;
#20;
```

```
        // XOR
        ALUOp = 4'b1000;
        #20;

        // BNEZ
        ALUOp = 4'b0111;
        #20;

        $stop;
    end
endmodule
```